# AISCI5000

# Foundation of Machine Learning and AI

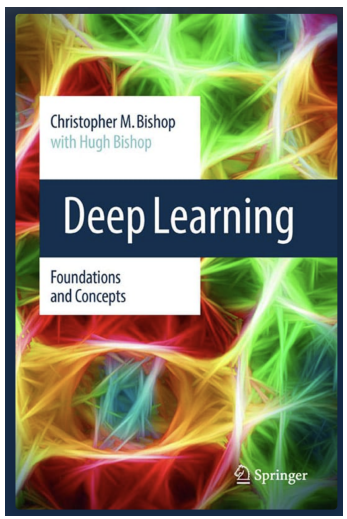
Single-Layer Networks: Regression

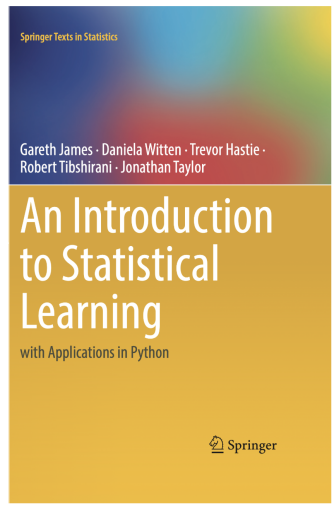
**Textbook**:

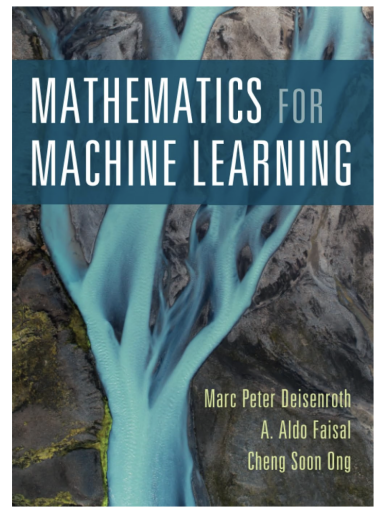
- C. M. Bishop, H. Bishop, *Deep Learning: Foundations and Concepts*, Ch. 1 & 4 (2024).

**Reference**:

- James, Witten, Hastie, Tibshirani, Taylor, *An Introduction to Statistical Learning*, Ch. 2 & 6 (2023).
- Deisenroth, Faisal, Ong, *Mathematics for Machine Learning*, (2020).
- 3blue1brown



# Outline

- 1.2 A Tutorial Example
- 2.3.4 The Probabilistic Perspective of Linear Regression
- 4.1 Linear Regression as a Single-Layer Neural Network
  - Basis Functions
  - Likelihood Function
  - Maximum Likelihood & Normal Equations
  - Geometry of Least Squares
  - Regularized Least Squares
- 4.2 Decision Theory for Regression
- 4.3 The Bias-Variance Trade-off

# 1.2 A Tutorial Example: Curve Fitting

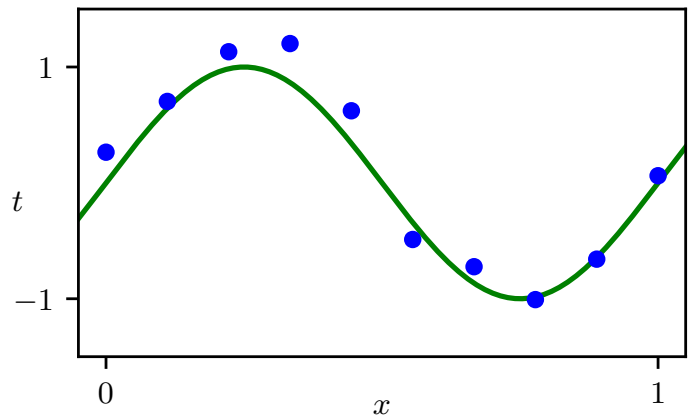
Many basic concepts and terms in machine learning can be illustrated by a simple regression problem: fitting a polynomial to a curve. This is an example of supervised learning, where we predict a target (output) variable $t$ given an input variable $x$.

- We have a training set with $N$ observations $\boldsymbol{x} = (x_1, \ldots, x_N)^T$, and the corresponding target values $\boldsymbol{t} = (t_1, \ldots, t_N)^T$.
- Our goal: Predict the target values $t$ for new, previously unseen input values $x$.

The ability to make accurate predictions on previously unseen input is a key objective in machine learning known as **generalization**.

Let us consider a training set with $N = 10$ data points:

- $x_n$ $(n = 1, ..., 10)$ uniformly spaced over $[0, 1]$.
- $t_n$ generated from $f(x) = \sin(2\pi x)$ plus a Gaussian noise $\epsilon \sim \mathcal{N}(0, \sigma^2)$: $t_n = \sin(2\pi x_n) + \epsilon_n$




Real-world data sets have an underlying regularity, which we wish to learn, but individual observations are corrupted by random noise from:

- Measurement errors
- Truly stochastic processes
- Variability from unobserved sources

Our task is to infer an entire continuous curve given only a finite number of data points. This task is further complicated by the random noise corrupting our observations—we need to distinguish the true underlying pattern, $\sin(2\pi x)$, from the noise, $\epsilon$.

Indeed, learning probabilities from data lies at the heart of machine learning and is a theme of our lectures.

We will begin with a simple approach, where we fit the observed training data using a polynomial:

$$y(x, \mathbf{w}) = w_0 + w_1 x + w_2 x^2 + \cdots + w_M x^M = \sum_{j=0}^{M} w_j x^j . \tag{1}$$

Here, $M$ is the order of the polynomial and $\mathbf{w} = (w_0, w_1, \ldots, w_M)^T$ represents the (column) vector of polynomial coefficients $w_0$, ..., $w_M$.

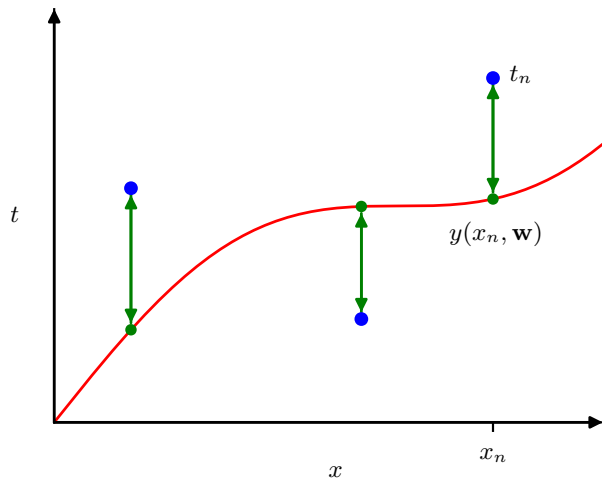
# Linear Models

- Although $y(x, \mathbf{w})$ is a non-linear function of the input variable $x$, it is a **linear function of the coefficients** $\mathbf{w}$.
- Functions linear in their parameters possess important analytical properties and represent the class of **linear models**.

# Sum-of-Squares Error (SSE)

The "fitting" of the polynomial (finding the parameter vector $\mathbf{w}$) to our data can be done by minimizing an error function that measures the misfit between predictions $y(x_n, \mathbf{w})$ and targets $t_n$. A widely used choice is the *sum-of-squares error function*:

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^{N} \{y(x_n, \mathbf{w}) - t_n\}^2 \tag{2}$$




- Geometrically, $E(\mathbf{w})$ is half the sum of squared vertical displacements between data points and the fitted polynomial.

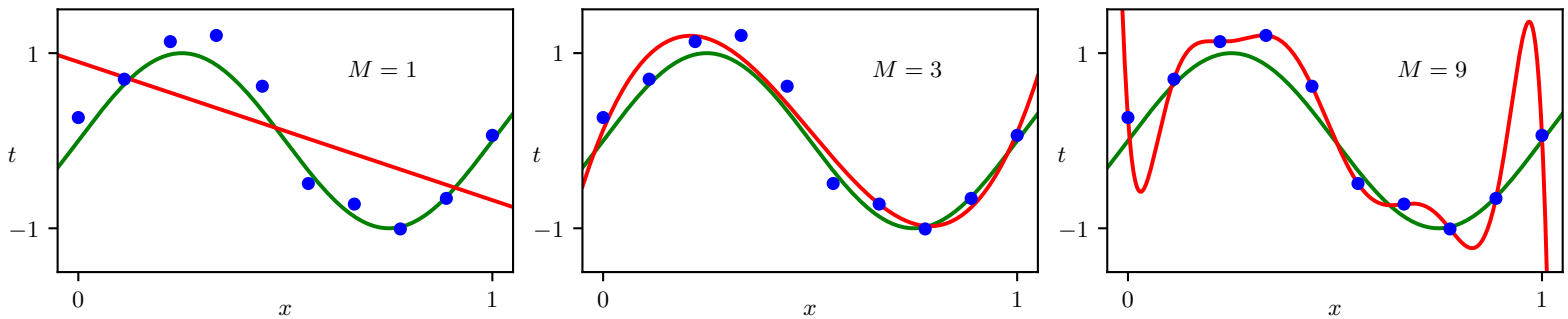
- $E(\mathbf{w}) = 0$ if and only if $y(x, \mathbf{w})$ passes exactly through every training data point.
- Because $E(\mathbf{w})$ is a quadratic function of the coefficients $\mathbf{w}$, its derivative with respect to $\mathbf{w}$ is linear in $\mathbf{w}$.
- Setting the derivative to zero yields a unique global minimum denoted by $\mathbf{w}^*$.
- The optimal solution $\mathbf{w}^*$ can be solved directly in closed form (we will derive this later), resulting in the final polynomial

$$y^*(x) = y(x, \mathbf{w}^*). \tag{3}$$

- Later we will show that the sum-of-squares form can be derived by maximizing the likelihood function under an assumed Gaussian noise.

# What $M$ should we choose?

Choosing the order $M$ of the polynomial controls model complexity (flexibility):




- $M = 0, 1$ **(Under-fitting)**: Fails to capture the underlying sinusoidal variation.
- $M = 3$: Provides an optimal fit to the underlying function $\sin(2\pi x)$.
- $M = 9$ **(Over-fitting)**: Passes through every point ($E(\mathbf{w}^*) = 0$) but oscillates wildly between points.

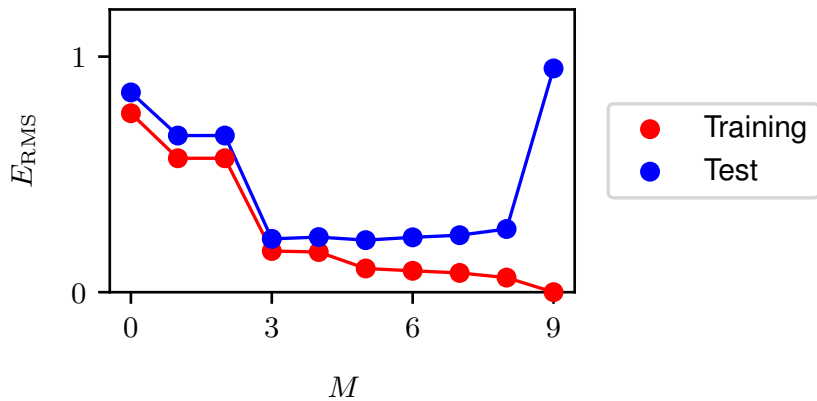
To evaluate performance across different model orders $M$ and dataset sizes, we introduce the root-mean-square (RMS) error:

$$E_{\text{RMS}} = \sqrt{\frac{2E(\mathbf{w}^*)}{N}} = \sqrt{\frac{1}{N}\sum_{n=1}^{N}\{y(x_n, \mathbf{w}^*) - t_n\}^2} \tag{4}$$

- Division by $N$ allows comparison of datasets of different sizes on an equal footing.
- The square root ensures $E_{\text{RMS}}$ is measured on the same scale (and in the same units) as the target variable $t$.
- $E_{\text{RMS}}$ is not what we minimize in training (MSE or SSE is).

# Over-Fitting

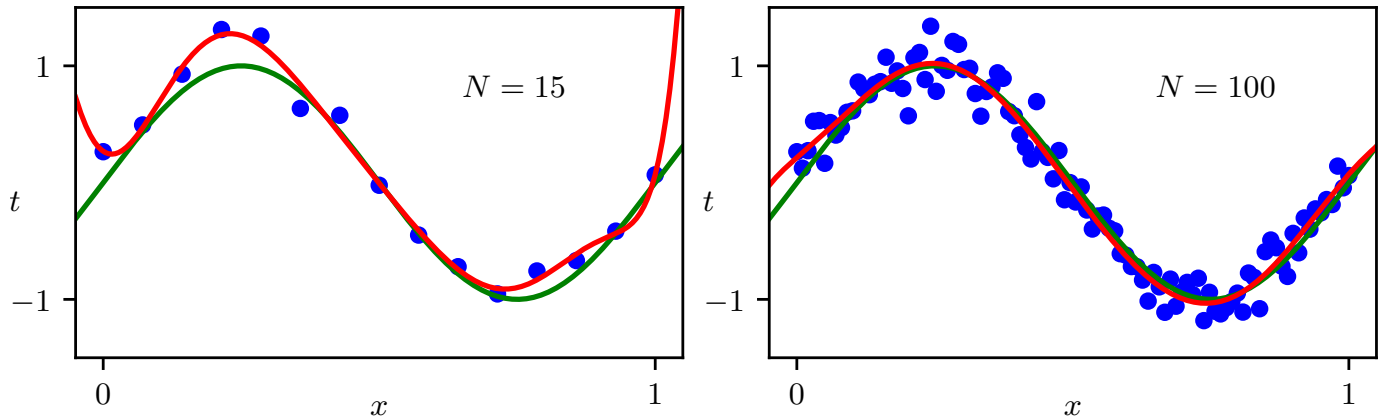
| | $M=0$ | $M=1$ | $M=3$ | $M=9$ |
|---|---|---|---|---|
| $w_0^\star$ | 0.11 | 0.90 | 0.12 | 0.26 |
| $w_1^\star$ | | −1.58 | 11.20 | −66.13 |
| $w_2^\star$ | | | −33.67 | 1,665.69 |
| $w_3^\star$ | | | 22.43 | −15,566.61 |
| $w_4^\star$ | | | | 76,321.23 |
| $w_5^\star$ | | | | −217,389.15 |
| $w_6^\star$ | | | | 370,626.48 |
| $w_7^\star$ | | | | −372,051.47 |
| $w_8^\star$ | | | | 202,540.70 |
| $w_9^\star$ | | | | −46,080.94 |




- As the polynomial order $M$ increases, the magnitudes of the coefficients $|\mathbf{w}^*|$ grow dramatically.
- For $M = 9$, coefficients develop huge positive and negative canceling values. The training error is zero but the test error is large.

The above observations suggest that flexible models can be finely tuned to the specific noise in the training set rather than the underlying trend.

For a fixed model complexity (e.g., $M = 9$), increasing the dataset size $N$ mitigates the over-fitting problem.




- Larger datasets allow us to fit more complex, flexible models.
- Classical heuristic: $N$ should be at least $5\times$ or $10\times$ the number of learnable parameters (Although modern deep learning routinely violates this heuristic by successfully training over-parameterized models).

# Regularization for Linear Regression

Regularization is a technique to control the over-fitting problem without arbitrarily limiting the number of parameters (e.g., $M$). As an example, we add a penalty term $E_W(\mathbf{w})$ to the error function:

$$\tilde{E}(\mathbf{w}) = \frac{1}{2}\sum_{n=1}^{N}\{y(x_n, \mathbf{w}) - t_n\}^2 + \frac{\lambda}{2}\|\mathbf{w}\|^2 \tag{5}$$

where $\|\mathbf{w}\|^2 \equiv \mathbf{w}^T\mathbf{w} = w_0^2 + w_1^2 + \cdots + w_M^2$.

- $\lambda$ controls regularization strength.
- $\tilde{E}(\mathbf{w})$ minimization still has a closed-form solution (to be derived later).

Terminology wise, we often refer to regularization methods forcing coefficients toward zero as **shrinkage** methods. In the context of neural networks, they are known as **weight decay**.

# Hyperparameters

- Parameters like the regularization coefficient $\lambda$ or the model order $M$ are examples of **hyperparameters**.
- Hyperparameters cannot be set by minimizing standard training error directly (since this will lead to $\lambda \to 0$ and an over-fitted model with large $M$).

The solution is to partition our data into:

- *Training Set*: Determines $\mathbf{w}$ for a given model (e.g., $M$).
- *Validation (Hold-Out) Set*: Selects hyperparameters (e.g., $\lambda, M$) based on the model that gives the lowest error.
- *Test Set*: Evaluates our final model's generalization performance.

# Cross-Validation

When data is limited, we wish to use as much of the available data as possible for training.

$S$-**Fold Cross-Validation**:

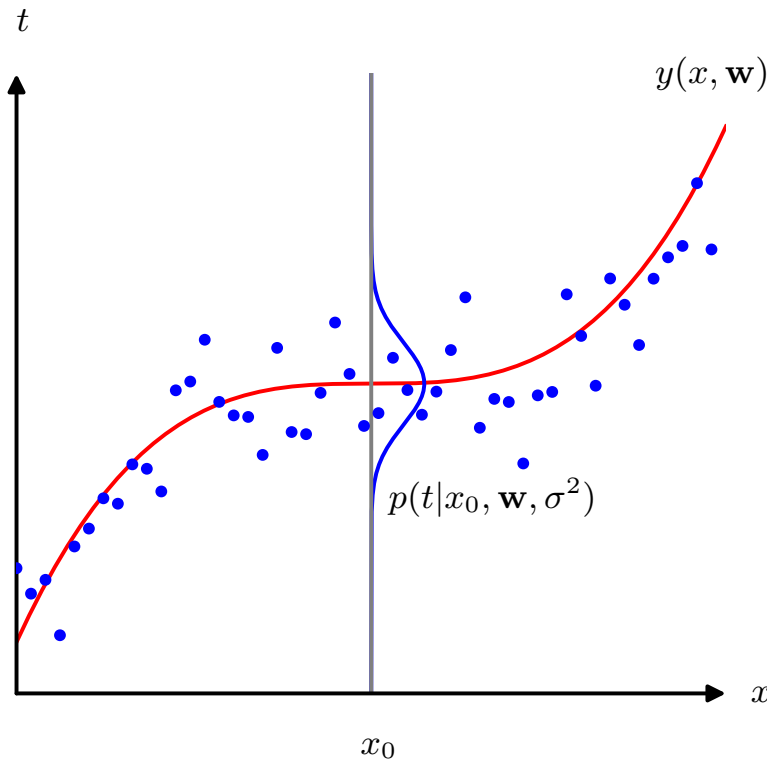
- Partition data into $S$ equal groups.
- Train on $S-1$ groups and evaluate on remaining held-out group.
- Repeat $S$ times and average performance scores across all runs.
- $S = N$ is called *Leave-One-Out cross-validation*.
- *Drawback*: $S$-fold cross-validation increases our computational cost by a factor of $S$, which may be computationally infeasible for massive deep networks.

# 2.3.4 The Probabilistic Perspective of Linear Regression

In our tutorial example of curve fitting, linear regression was formulated via error minimization (e.g., SSE). We will now revisit this problem from a probabilistic perspective. This perspective allows us to better understand:

- Why specific error functions (sum-of-squares) are used.
- How target uncertainty can be explicitly modeled.
- The statistical foundation for regularization.

Again, our goal is to predict $t$ for any new input $x$, given $N$ input observations $\boldsymbol{x} = (x_1, \ldots, x_N)^T$ and the corresponding target values $\boldsymbol{t} = (t_1, \ldots, t_N)^T$.




We express the uncertainty over the target variable $t$ by modeling it with a conditional Gaussian distribution:

$$p(t \mid x, \mathbf{w}, \sigma^2) = \mathcal{N}(t \mid y(x, \mathbf{w}), \sigma^2) \tag{6}$$

where $y(x, \mathbf{w}) = \sum_{j=0}^{M} w_j x^j$ represents the mean of the conditional Gaussian and $\sigma^2$ is the variance of the additive Gaussian noise.

# Density Estimation

Now our task becomes to estimate $\mathbf{w}$ and $\sigma^2$ from a set of training data. This is known as *density estimation*. In general, such a problem is fundamentally ill-posed, because there are infinitely many probability distributions that could have given rise to the observed finite data set.

- By assuming that our target variable $t$ follows a Gaussian distribution conditioned on the input variable $x$, we introduce a constraint to the density estimation problem and can obtain a well-defined solution.

Data points drawn independently from the same distribution are said to be **independent and identically distributed (i.i.d.)**.

- Because our data set is (assumed to be) i.i.d., we can apply the product rule to obtain the likelihood function.

# Likelihood Function

The joint probability of two independent events is the product of the marginal probabilities of each event. Assuming our training data points are i.i.d. from $\mathcal{N}(t \mid y(x, \mathbf{w}), \sigma^2)$, we obtain the likelihood function as:

$$p(\boldsymbol{t} \mid \boldsymbol{x}, \mathbf{w}, \sigma^2) = \prod_{n=1}^{N} \mathcal{N}(t_n \mid y(x_n, \mathbf{w}), \sigma^2) \tag{7}$$

- The above function is a conditional probability distribution (parameterized by $\mathbf{w}$ and $\sigma^2$) of $\boldsymbol{t}$ given $\boldsymbol{x}$.
- It is a likelihood function when viewed as a function of $\mathbf{w}$ and $\sigma^2$ (both $\boldsymbol{t}$ and $\boldsymbol{x}$ are known).

Taking its natural logarithm, we obtain the *log likelihood*:

$$\ln p(\boldsymbol{t} \mid \boldsymbol{x}, \mathbf{w}, \sigma^2) = -\frac{1}{2\sigma^2} \sum_{n=1}^{N} \{y(x_n, \mathbf{w}) - t_n\}^2 - \frac{N}{2} \ln \sigma^2 - \frac{N}{2} \ln(2\pi) \tag{8}$$

To determine the **maximum likelihood (ML) solution** for the weight parameters $\mathbf{w}_{ML}$, we maximize $\ln p(\boldsymbol{t} \mid \boldsymbol{x}, \mathbf{w}, \sigma^2)$ with respect to $\mathbf{w}$:

- The terms $-\frac{N}{2} \ln \sigma^2$ and $-\frac{N}{2} \ln(2\pi)$ do not depend on $\mathbf{w}$ and can be dropped.
- Scaling by the positive constant factor $\sigma^2$ does not alter the location of the optimum.
- Maximizing the log likelihood is equivalent to minimizing the negative log likelihood.

Our derivation shows that maximizing likelihood with respect to $\mathbf{w}$ is equivalent to minimizing the *sum-of-squares error function*:

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^{N} \{y(x_n, \mathbf{w}) - t_n\}^2 \tag{9}$$

# Predictive Distribution

Once $\mathbf{w}_{ML}$ is found, we can also use maximum likelihood to estimate the noise variance parameter $\sigma^2$.

Maximizing $\ln p(\boldsymbol{t} \mid \boldsymbol{x}, \mathbf{w}, \sigma^2)$ with respect to $\sigma^2$ yields:

$$\sigma^2_{ML} = \frac{1}{N} \sum_{n=1}^{N} \{y(x_n, \mathbf{w}_{ML}) - t_n\}^2 \tag{10}$$

Having obtained the parameters $\mathbf{w}_{ML}$ and $\sigma^2_{ML}$, we can make predictions for new inputs $x$. Instead of merely a point estimate, we now have a complete predictive distribution over $t$:

$$p(t \mid x, \mathbf{w}_{ML}, \sigma^2_{ML}) = \mathcal{N}(t \mid y(x, \mathbf{w}_{ML}), \sigma^2_{ML}) \tag{11}$$

# 4.1 Linear Regression as a Single-Layer Neural Network

Expanding our earlier discussions, we now consider a slightly more general regression problem: predicting a target variable $t$ given a $D$-dimensional input vector $\mathbf{x} = (x_1, \ldots, x_D)^T$.

$$y(\mathbf{x}, \mathbf{w}) = w_0 + w_1 x_1 + \cdots + w_D x_D \tag{12}$$

where $\mathbf{w} = (w_0, w_1, \ldots, w_D)^T$ are learnable parameters.

- Note that $\mathbf{x} = (x_1, \ldots, x_D)^T$ represents a single observation of a $D$-dimensional vector-valued variable. It is not to be confused with the vector $\boldsymbol{x} = (x_1, \ldots, x_N)^T$ representing $N$ observations of a scalar variable. Sometimes the latter is given as a row vector instead: $\boldsymbol{x} = (x_1, \ldots, x_N)$.

Because the above model is linear with respect to $x_i$, it cannot capture non-linear patterns or high-dimensional feature interactions. We can extend our model by considering linear combinations of *basis functions* $\phi_j(\mathbf{x})$ of the input variables.

- *Polynomial Basis* (global):

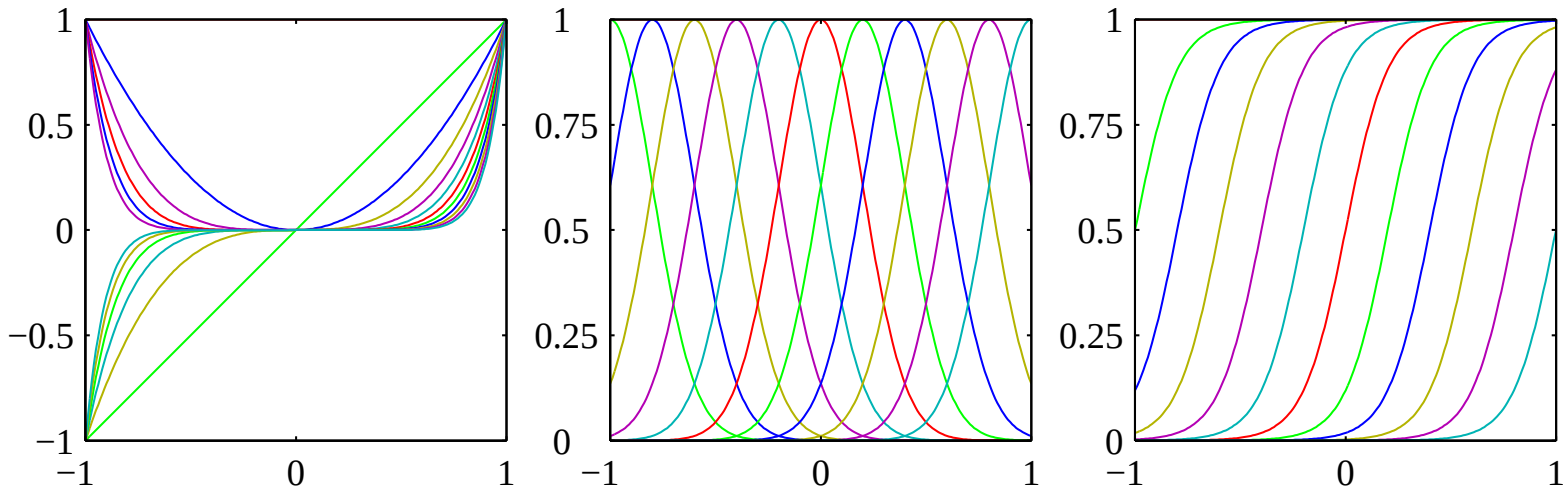
$$\phi_j(x) = x^j$$

- *Gaussian Basis* (localized):

$$\phi_j(x) = \exp\left(-\frac{(x-\mu_j)^2}{2s^2}\right)$$

- *Sigmoidal Basis* (step-like):

$$\phi_j(x) = \sigma\left(\frac{x-\mu_j}{s}\right), \quad \text{where } \sigma(a) = \frac{1}{1+e^{-a}}$$



Building upon the **basis functions** $\phi_j(\mathbf{x})$, our model becomes:

$$y(\mathbf{x}, \mathbf{w}) = w_0 + \sum_{j=1}^{M-1} w_j \phi_j(\mathbf{x}) = \sum_{j=0}^{M-1} w_j \phi_j(\mathbf{x}) = \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}) \tag{13}$$

- $\phi_0(\mathbf{x}) = 1$ is a dummy basis function, introduced to "absorb" the *bias* parameter $w_0$ into the parameter vector $\mathbf{w}$.
- $\phi_j(\mathbf{x})$ takes the entire input vector $\mathbf{x}$ as an argument and can "spit out" any linear or non-linear combinations of its elements.
- $\mathbf{x}$ and $\boldsymbol{\phi}(\mathbf{x})$ need not have the same dimension ($D$ *vs* $M$).

Note that we still have a **linear model**—our model is linear in $\mathbf{w}$.

Now we construct the likelihood function as before. We again assume that our target variable $t$ is given by a deterministic function $y(\mathbf{x}, \mathbf{w})$ with an additive Gaussian noise $\epsilon \sim \mathcal{N}(0, \sigma^2)$: $t = y(\mathbf{x}, \mathbf{w}) + \epsilon$. The conditional distribution of $t$ is therefore:

$$p(t|\mathbf{x}, \mathbf{w}, \sigma^2) = \mathcal{N}(t|\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}), \sigma^2) \tag{14}$$

Given the input $\mathbf{X}$ and output $\boldsymbol{t} = (t_1, \ldots, t_N)^T$, we again multiply the individual observations' conditional distributions (based on their i.i.d. nature) to find the likelihood function:

$$p(\boldsymbol{t}|\mathbf{X}, \mathbf{w}, \sigma^2) = \prod_{n=1}^{N} \mathcal{N}(t_n|\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n), \sigma^2) \tag{15}$$

- $\mathbf{X}$ is an $N$ by $D$ matrix, with each row representing $\mathbf{x}_n^T$, the $n$-th observation of the $D$-dimensional vector-valued input variable.

# Maximum Likelihood Solution

Taking log-likelihood of $p(\mathbf{t}|\mathbf{X}, \mathbf{w}, \sigma^2)$:

$$\ln p(\mathbf{t}|\mathbf{X}, \mathbf{w}, \sigma^2) = -\frac{N}{2} \ln \sigma^2 - \frac{N}{2} \ln(2\pi) - \frac{1}{\sigma^2} E_D(\mathbf{w}) \tag{16}$$

where the sum-of-squares error is:

$$E_D(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^{N} \{t_n - \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n)\}^2 \tag{17}$$

Again, we see that maximizing log-likelihood with respect to $\mathbf{w}$ under Gaussian noise assumption is **equivalent** to minimizing sum-of-squares error $E_D(\mathbf{w})$.

$$\nabla_{\mathbf{w}} \ln p = \frac{1}{\sigma^2} \sum_{n=1}^{N} \{t_n - \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n)\} \boldsymbol{\phi}(\mathbf{x}_n)^T = 0 \tag{18}$$

Setting derivative of log-likelihood with respect to $\mathbf{w}$ to zero as shown above, we will obtain a set of equations known as the normal equations.

# Design Matrix

Before we proceed to derive the normal equations, let us define $\mathbf{\Phi}$, an $N \times M$ matrix called the Design Matrix. Its entries are given by $\Phi_{nj} = \phi_j(\mathbf{x}_n)$:

$$\mathbf{\Phi} = \begin{bmatrix} \phi_0(\mathbf{x}_1) & \phi_1(\mathbf{x}_1) & \cdots & \phi_{M-1}(\mathbf{x}_1) \\ \phi_0(\mathbf{x}_2) & \phi_1(\mathbf{x}_2) & \cdots & \phi_{M-1}(\mathbf{x}_2) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_0(\mathbf{x}_N) & \phi_1(\mathbf{x}_N) & \cdots & \phi_{M-1}(\mathbf{x}_N) \end{bmatrix} \tag{19}$$

- Row $n$ of $\mathbf{\Phi}$ is an $M$-dimensional row vector representing the basis functions applied on the $n$-th observation of the input (recall that the input $\mathbf{x}_n$ is itself a $D$-dimensional vector):

$$\boldsymbol{\phi}(\mathbf{x}_n)^T = \big(\phi_0(\mathbf{x}_n), \phi_1(\mathbf{x}_n), ..., \phi_{M-1}(\mathbf{x}_n)\big) \tag{20}$$

- Column $j$ of $\mathbf{\Phi}$ is an $N$-dimensional column vector representing the $j$-th basis function applied on all $N$ observations of the input. From hereon, we will define it as $\boldsymbol{\varphi}_j$:

$$\boldsymbol{\varphi}_j = \big(\phi_j(\mathbf{x}_1), \phi_j(\mathbf{x}_2), \ ..., \ \phi_j(\mathbf{x}_N)\big)^T \tag{21}$$

The transpose of the design matrix is

$$\mathbf{\Phi}^T = \begin{bmatrix} \phi_0(\mathbf{x}_1) & \phi_0(\mathbf{x}_2) & \cdots & \phi_0(\mathbf{x}_N) \\ \phi_1(\mathbf{x}_1) & \phi_1(\mathbf{x}_2) & \cdots & \phi_1(\mathbf{x}_N) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_{M-1}(\mathbf{x}_1) & \phi_{M-1}(\mathbf{x}_2) & \cdots & \phi_{M-1}(\mathbf{x}_N) \end{bmatrix} \tag{22}$$

It is straightforward to verify that

- $\mathbf{\Phi} = \big[\boldsymbol{\varphi}_0, \boldsymbol{\varphi}_1, \ ..., \ \boldsymbol{\varphi}_{M-1}\big]$
- $\mathbf{\Phi}^T = \big[\boldsymbol{\phi}(\mathbf{x}_1), \boldsymbol{\phi}(\mathbf{x}_2), \ ..., \ \boldsymbol{\phi}(\mathbf{x}_N)\big]$

Based on the design matrix, we can rewrite the SSE as

$$E_D(\mathbf{w}) = \frac{1}{2}(\mathbf{t} - \mathbf{\Phi w})^T(\mathbf{t} - \mathbf{\Phi w}) \tag{23}$$

Verify yourself that the above is consistent with Eq. 17.

Expanding this equation, we have:

$$E_D(\mathbf{w}) = \frac{1}{2}(\mathbf{t}^T\mathbf{t} - \mathbf{w}^T\mathbf{\Phi}^T\mathbf{t} - \mathbf{t}^T\mathbf{\Phi w} + \mathbf{w}^T\mathbf{\Phi}^T\mathbf{\Phi w}) \tag{24}$$

Verify yourself that every item in the parenthesis is a scalar.

The second and third items are transposes of each other and are therefore equal (since they are both scalars). Therefore,

$$\begin{aligned}\nabla_{\mathbf{w}} E_D(\mathbf{w}) &= \frac{1}{2}\left[\nabla_{\mathbf{w}}\left(\mathbf{t}^T\mathbf{t}\right) - \nabla_{\mathbf{w}}\left(2\mathbf{w}^T\mathbf{\Phi}^T\mathbf{t}\right) + \nabla_{\mathbf{w}}\left(\mathbf{w}^T\mathbf{\Phi}^T\mathbf{\Phi w}\right)\right] \\ &= \frac{1}{2}\left(-2\mathbf{\Phi}^T\mathbf{t} + 2\mathbf{\Phi}^T\mathbf{\Phi w}\right) \\ &= \mathbf{\Phi}^T\mathbf{\Phi w} - \mathbf{\Phi}^T\mathbf{t}\end{aligned} \tag{25}$$

# Normal Equations

Setting $\nabla_{\mathbf{w}} E_D(\mathbf{w})$ to zero yields the **Normal Equations**:

$$\mathbf{\Phi}^T \mathbf{\Phi} \mathbf{w} = \mathbf{\Phi}^T \mathbf{t} \tag{26}$$

Solving these equations yields the maximum likelihood solution for $\mathbf{w}$:

$$\mathbf{w}_{\mathsf{ML}} = (\mathbf{\Phi}^T \mathbf{\Phi})^{-1} \mathbf{\Phi}^T \mathbf{t} = \mathbf{\Phi}^\dagger \mathbf{t} \tag{27}$$

where $\mathbf{\Phi}^\dagger = (\mathbf{\Phi}^T \mathbf{\Phi})^{-1} \mathbf{\Phi}^T$ is known as the *Moore-Penrose pseudo-inverse*.

As before, we also obtain the maximum likelihood solution for $\sigma^2$:

$$\sigma^2_{\mathsf{ML}} = \frac{1}{N} \sum_{n=1}^{N} \{t_n - \mathbf{w}^T_{\mathsf{ML}} \boldsymbol{\phi}(\mathbf{x}_n)\}^2 \tag{28}$$

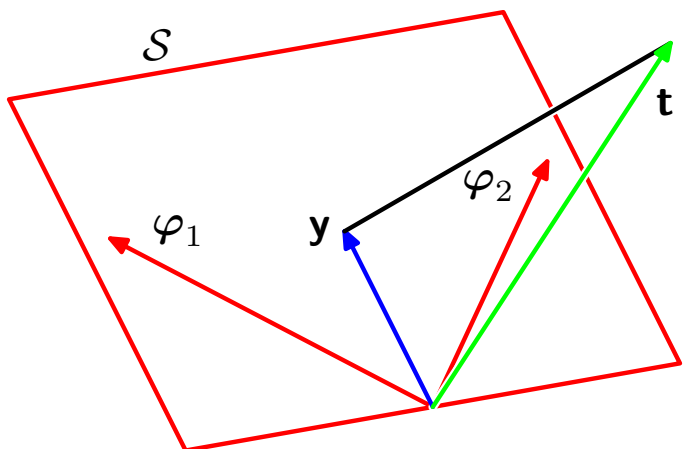
Question: When is $\mathbf{\Phi}^T \mathbf{\Phi}$ invertible?

# Geometry of Least Squares

Geometrically, we can view the regression problem from a vector projection perspective:

- The target vector $\boldsymbol{t} \in \mathbb{R}^N$.
- The column vectors $\boldsymbol{\varphi}_j$ of the design matrix $\boldsymbol{\Phi}$ are also in $\mathbb{R}^N$.

Linear regression can be understood as asking: can $\boldsymbol{t}$ be expressed as a linear combination of $\boldsymbol{\varphi}_0, \boldsymbol{\varphi}_1, \ldots, \boldsymbol{\varphi}_{M-1}$? If not, what is the closest (best) approximation to $\boldsymbol{t}$ we can obtain by linearly combining $\boldsymbol{\varphi}_0, \boldsymbol{\varphi}_1, \ldots, \boldsymbol{\varphi}_{M-1}$?



- The model output vector $\boldsymbol{y} = \boldsymbol{\Phi}\mathbf{w}$ is in the subspace $\mathcal{S}$ spanned by $\varphi_0$, ..., $\varphi_{M-1}$ (Why?).
- To find the closest approximation to $\boldsymbol{t}$ in $\mathcal{S}$, we demand that the error vector $\mathbf{y} - \mathbf{t}$ must be orthogonal to $\mathcal{S}$:

$$\boldsymbol{\Phi}^T(\mathbf{y} - \mathbf{t}) = 0 \implies \boldsymbol{\Phi}^T(\boldsymbol{\Phi}\mathbf{w} - \mathbf{t}) = 0 \tag{29}$$

- We recover the *Normal Equations*:

$$\boldsymbol{\Phi}^T\boldsymbol{\Phi}\mathbf{w} = \boldsymbol{\Phi}^T\mathbf{t} \tag{30}$$

Now we see that our maximum likelihood solution $\mathbf{w}_{\mathsf{ML}}$ can also be found from the orthogonal projection of the target vector $\mathbf{t}$ onto $\mathcal{S}$: $\mathbf{w}_{\mathsf{ML}}$ is the list of weights we need to linearly combine $\varphi_0, \varphi_1$, ..., $\varphi_{M-1}$ into this orthogonal projection.

# Sequential Learning

The least-squares problem is a rare example that can be solved in closed form. However, in practice, directly solving for $\mathbf{w}_{\text{ML}}$ via $(\mathbf{\Phi}^T\mathbf{\Phi})^{-1}\mathbf{\Phi}^T\mathbf{t}$ can lead to numerical difficulties. Furthermore, batch calculations using all observations at once is computationally expensive.

In Chapter 7, we will learn *Stochastic Gradient Descent (SGD)*, where we update the parameter vector after presenting each data point $n$:

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \nabla E_n \tag{31}$$

For sum-of-squares error $E_n = \frac{1}{2}(t_n - \mathbf{w}^T\boldsymbol{\phi}_n)^2$, this yields the *Least-Mean-Squares (LMS)* algorithm:

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} + \eta(t_n - \mathbf{w}^{(\tau)T}\boldsymbol{\phi}_n)\boldsymbol{\phi}_n \tag{32}$$

where $\eta$ is the learning rate.

# Regularized Least Squares

Before we end our discussion on the technical details of linear regression, we will consider two more aspects: We examine how regularization changes our solution and briefly discuss the case of multiple outputs.

Combining our sum-of-squares error with a regularization penalty term $E_W(\mathbf{w})$, we have:

$$E(\mathbf{w}) = E_D(\mathbf{w}) + \lambda E_W(\mathbf{w}) \tag{33}$$

Using $L_2$ regularization $E_W(\mathbf{w}) = \frac{1}{2}\mathbf{w}^T\mathbf{w}$ gives:

$$E(\mathbf{w}) = \frac{1}{2}\sum_{n=1}^{N}\{t_n - \mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}_n)\}^2 + \frac{\lambda}{2}\mathbf{w}^T\mathbf{w} \tag{34}$$

Setting its gradient to zero yields the regularized solution:

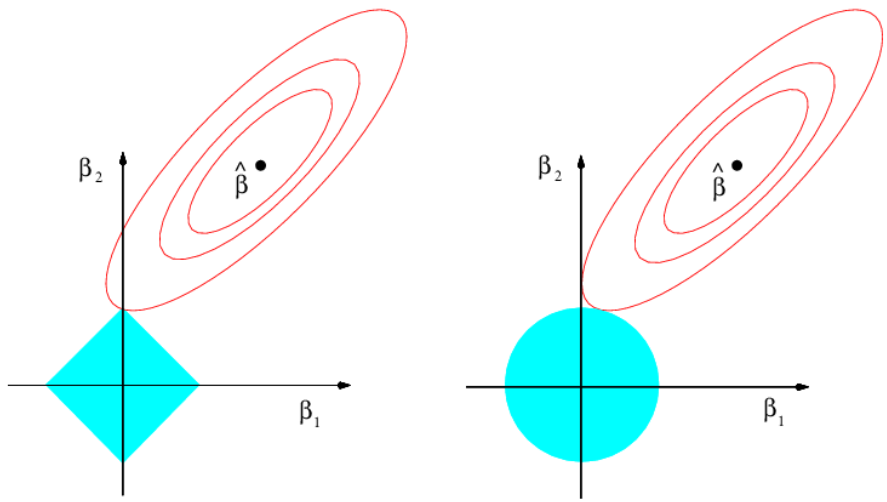
$$\mathbf{w} = (\lambda\mathbf{I} + \boldsymbol{\Phi}^T\boldsymbol{\Phi})^{-1}\boldsymbol{\Phi}^T\boldsymbol{t} \tag{35}$$

The regularized solution has clear numerical advantages:

- The matrix $\mathbf{\Phi}^T\mathbf{\Phi}$ can be singular or ill-conditioned if features are collinear or $M > N$.
- Adding $\lambda\mathbf{I}$ guarantees positive-definiteness and invertibility.

Apart from $L_2$ regularization, we may consider other weight penalties:

- Generalized penalty: $\frac{\lambda}{2}\sum|w_j|^q$.
- $q = 1$ corresponds to $L_1$ or **Lasso regularization**, which encourages sparse parameter solutions ($w_j = 0$).



# Multiple Target Variables

Finally, we consider predicting $K > 1$ target variables $\mathbf{t} = (t_1, \ldots, t_K)^T$:

$$\mathbf{y}(\mathbf{x}, \mathbf{W}) = \mathbf{W}^T \boldsymbol{\phi}(\mathbf{x}) \tag{36}$$

where $\mathbf{W}$ is an $M \times K$ weight matrix.

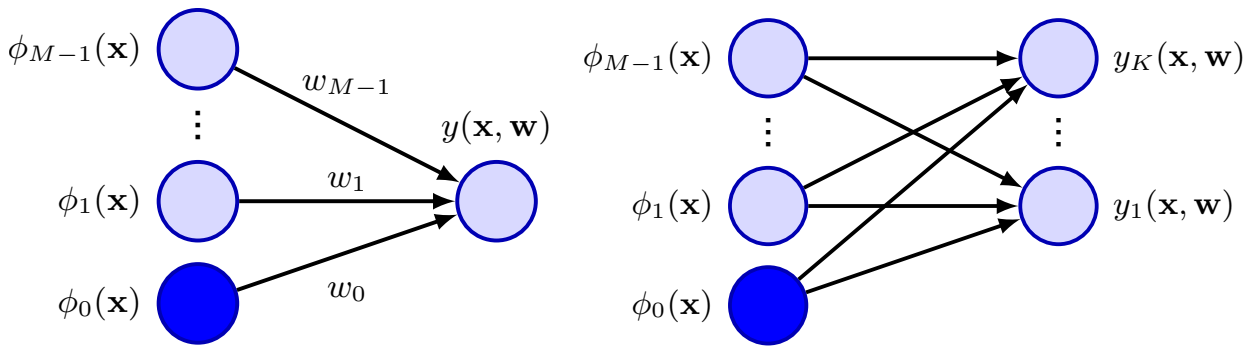
Assuming isotropic Gaussian noise $p(\mathbf{t}|\mathbf{x}, \mathbf{W}, \sigma^2) = \mathcal{N}(\mathbf{t}|\mathbf{W}^T\boldsymbol{\phi}(\mathbf{x}), \sigma^2\mathbf{I})$, we can follow previous steps to obtain the maximum likelihood solution:

$$\mathbf{W}_{\mathsf{ML}} = (\boldsymbol{\Phi}^T\boldsymbol{\Phi})^{-1}\boldsymbol{\Phi}^T\mathbf{T} = \boldsymbol{\Phi}^\dagger\mathbf{T} \tag{37}$$

where $\mathbf{T}$ is $N \times K$ target matrix.

Linear regression can be viewed as a single-layer neural network:

- Each basis function $\phi_j(\mathbf{x})$ represents an input node.
- The input nodes are connected to the output node $y(\mathbf{x}, \mathbf{w})$ (or $y_1(\mathbf{x}, \mathbf{w})$, ..., $y_K(\mathbf{x}, \mathbf{w})$) via a single layer of weights $w_j$.




The basis functions $\boldsymbol{\phi}(\mathbf{x})$ used in linear regression are hand-crafted. As we will learn in later lectures, deep learning replaces fixed basis functions with multiple parameter-dependent layers and learns non-linear representations and transformations directly from raw data.

# 4.2 Decision Theory for Regression

In the remainder of this lecture, we use linear regression as the framework to introduce two key concepts in machine learning.

The linear regression procedure can be divided into two explicit stages:

1. *Inference*: We use training data to model the conditional distribution $p(t|\mathbf{x})$.
2. *Decision*: We use the modelled $p(t|\mathbf{x})$ to determine a specific prediction for $t$.

The decision stage may appear "hidden" in our discussions thus far—this is because while conceptually it is the second stage, it can be (and in this case is) done first, *i.e.*, before the training starts.

We will use $f(\mathbf{x})$ to denote the specific prediction for $t$, based on a given input $\mathbf{x}$ and our predictive distribution $p(t|\mathbf{x}, \mathbf{w}_{ML}, \sigma^2_{ML})$. Since our prediction may not equal the true value of $t$, by using $f(\mathbf{x})$ as the prediction we incur some penalty (cost or loss). We represent this using a loss function $L(t, f(\mathbf{x}))$.

It is natural to set our goal to minimizing the average or **Expected Loss**:

$$\mathbb{E}[L] = \iint L(t, f(\mathbf{x}))p(\mathbf{x}, t)\, d\mathbf{x}\, dt \tag{38}$$

Note that this is a double integral of the joint distribution over both input ($\mathbf{x}$) and output ($t$).

Although we do not know the true $p(\mathbf{x}, t)$, we can still deduce certain properties of the optimal $f(\mathbf{x})$.

A common choice for $L(t, f(\mathbf{x}))$ is the **Squared Loss**:

$$L(t, f(\mathbf{x})) = \{f(\mathbf{x}) - t\}^2 \tag{39}$$

This gives:

$$\mathbb{E}[L] = \iint \{f(\mathbf{x}) - t\}^2 p(\mathbf{x}, t)\, d\mathbf{x}\, dt \tag{40}$$

To find $f(\mathbf{x})$ that minimizes $\mathbb{E}[L]$, we use calculus of variations, setting

$$\frac{\delta \mathbb{E}[L]}{\delta f(\mathbf{x})} = 0 \tag{41}$$

which leads to

$$2 \int \{f(\mathbf{x}) - t\} p(\mathbf{x}, t)\, dt = 0 \implies f^*(\mathbf{x}) = \frac{\int t p(\mathbf{x}, t) dt}{p(\mathbf{x})} = \int t p(t|\mathbf{x}) dt \tag{42}$$

By definition, the R.H.S. of the last equation is the conditional mean of $t$ at the given $\mathbf{x}$. Therefore, the $f(\mathbf{x})$ that minimizes $L(t, f(\mathbf{x}))$ is

$$f^*(\mathbf{x}) = \mathbb{E}_t[t|\mathbf{x}] \tag{43}$$

In summary, if we choose a squared loss function, the optimal prediction function is $\mathbb{E}_t[t|\mathbf{x}]$.

In our first linear regression problem, the target variable $t$ is assumed to follow a Gaussian distribution conditioned on the input variable $\mathbf{x}$. The symmetry of the Gaussian distribution dictates that $\mathbb{E}_t[t|\mathbf{x}] = y(\mathbf{x}, \mathbf{w}_{ML})$. Therefore, we should use $y(\mathbf{x}, \mathbf{w}_{ML})$ as the prediction function according to decision theory. And this is indeed what we have been doing all along.

# 4.3 The Bias-Variance Trade-off

Before we proceed with the final part of this lecture, it should be emphasized that $p(\mathbf{x}, t)$ in Eq. 40 refers to the true joint distribution of $t$ and $\mathbf{x}$. Denoting the true conditional mean $\mathbb{E}_t[t|\mathbf{x}]$ as $h(\mathbf{x})$, we can rewrite the expected loss as:

$$\begin{aligned}\mathbb{E}[L] &= \iint \{f(\mathbf{x}) - h(\mathbf{x}) + h(\mathbf{x}) - t\}^2 p(\mathbf{x}, t)\, d\mathbf{x}\, dt \\ &= \int \{f(\mathbf{x}) - h(\mathbf{x})\}^2 p(\mathbf{x})\, d\mathbf{x} + \iint \{h(\mathbf{x}) - t\}^2 p(\mathbf{x}, t)\, d\mathbf{x}\, dt\end{aligned} \tag{44}$$

Notice that the cross-terms have vanished.

We can further analyze the integrand of the first term: let $f(\mathbf{x}; \mathcal{D})$ represent the prediction function learned from data set $\mathcal{D}$. A second round of training using a different data set may give a different function: e.g., in linear regression, $f(\mathbf{x}) = y(\mathbf{x}, \mathbf{w})$, where $\mathbf{w}$ may change when different training data are used.

Let $\mathbb{E}_{\mathcal{D}}[f(\mathbf{x};\mathcal{D})]$ denote the expectation of all such learned functions, we have

$$\begin{aligned}\{f(\mathbf{x};\mathcal{D})-h(\mathbf{x})\}^2 =&\{f(\mathbf{x};\mathcal{D})-\mathbb{E}_{\mathcal{D}}[f(\mathbf{x};\mathcal{D})]\}^2\\ &+\{\mathbb{E}_{\mathcal{D}}[f(\mathbf{x};\mathcal{D})]-h(\mathbf{x})\}^2\\ &+2\{f(\mathbf{x};\mathcal{D})-\mathbb{E}_{\mathcal{D}}[f(\mathbf{x};\mathcal{D})]\}\{\mathbb{E}_{\mathcal{D}}[f(\mathbf{x};\mathcal{D})]-h(\mathbf{x})\}\end{aligned} \tag{45}$$

Taking expectation over datasets $\mathcal{D}$, we see that the cross-terms vanish again. Thus, we can decompose the total expected loss as:

$$\text{Expected Loss} = (\text{Bias})^2 + \text{Variance} + \text{Noise} \tag{46}$$

where

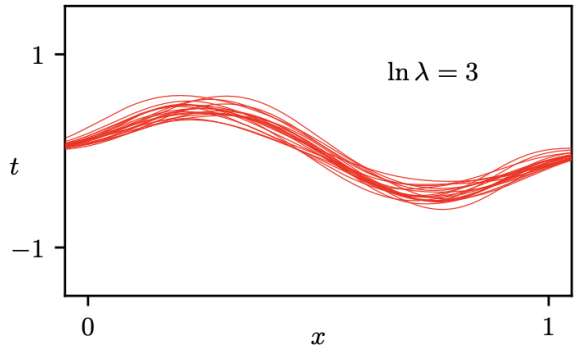
- $(\text{Bias})^2 = \int\{\mathbb{E}_{\mathcal{D}}[f(\mathbf{x};\mathcal{D})]-h(\mathbf{x})\}^2 p(\mathbf{x})d\mathbf{x}$
  - Misfit between average prediction and true mean.

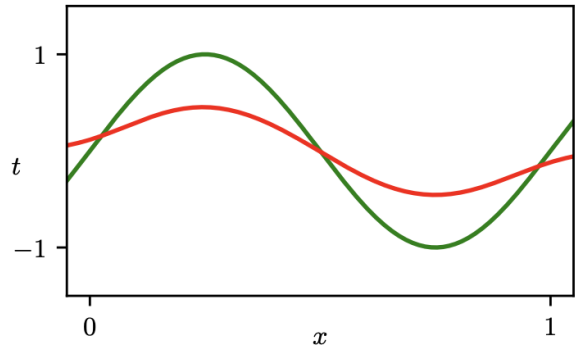
- Variance $= \int \mathbb{E}_{\mathcal{D}}[\{f(\mathbf{x};\mathcal{D}) - \mathbb{E}_{\mathcal{D}}[f(\mathbf{x};\mathcal{D})]\}^2]p(\mathbf{x})d\mathbf{x}$
  - Sensitivity of model prediction to particular dataset $\mathcal{D}$.
- Noise $= \int\int \{h(\mathbf{x}) - t\}^2 p(\mathbf{x}, t)d\mathbf{x}dt$
  - Irreducible variance in target distribution.

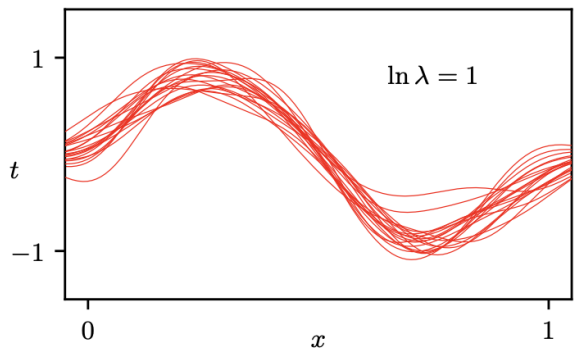
The noise term reflects the intrinsic variability of our data and cannot be reduced by simply increasing the amount of observations (However, as discussed in lecture 2, this noise may be reduced if observations of additional input variables are collected).

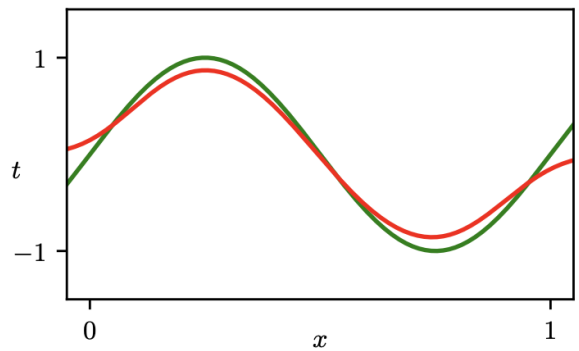
We use the sinusoidal data set to demonstrate the bias-variance trade-off:

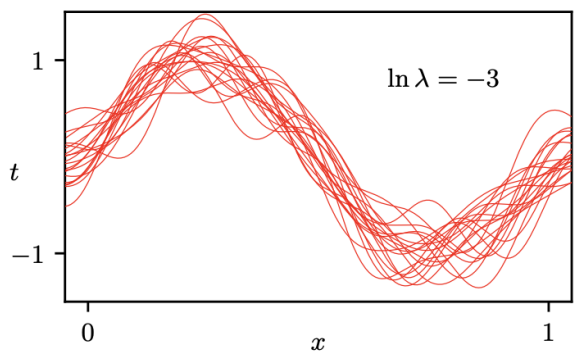
- We independently generate 100 data sets, each containing $N = 25$ data points from $h(x) = \sin(2\pi x)$.
- For each data set, we fit a model with $M = 24$ Gaussian basis functions along with a constant bias (a total of 25 parameters).
- We minimize the regularized error as given in Eq. 34.

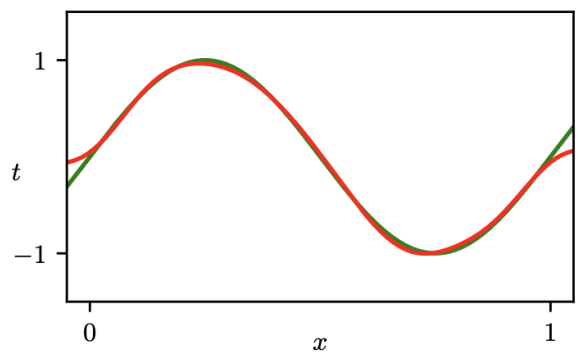

ln λ = 3
1
t
−1
0
x
1


1
t
−1
0
x
1


ln λ = 1
1
t
−1
0
x
1


1
t
−1
0
x
1


ln λ = −3
1
t
−1
0
x
1


1
t
−1
0
x
1

# Impact of Regularization ($\lambda$)

The above example demonstrates the *bias-variance trade-off* and how the regularization parameter $\lambda$ controls the effective model complexity:

- Large $\lambda$ (High Regularization):
  - Model is constrained and rigid $\Longrightarrow$ *High Bias, Low Variance*.
  - Leads to under-fitting.
- Small $\lambda$ (Low Regularization):
  - Model is highly flexible and sensitive to noise $\Longrightarrow$ *Low Bias, High Variance*.
  - Leads to over-fitting.

We will have more discussions on decision theory and bias-variance trade-off in the remainder of this course.